Problem G. 贪吃蛇

这是一道交互题。请注意在每次输出后刷新输出缓冲区。您可以使用以下方式刷新输出：

- 在 C/C++ 中使用 `fflush(stdout)` 或 `cout.flush()`；
- 在 Java 和 Kotlin 中使用 `System.out.flush()`；
- 在 Python 中使用 `sys.stdout.flush()`。

您正在一个 $n \times m$ 的网格上玩经典的贪吃蛇游戏。行从上到下编号为 1 到 n，列从左到右编号为 1 到 m。我们用 (i, j) 表示第 i 行第 j 列的格子。

蛇可以用一系列坐标对 $(x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)$ 表示其身体所在的位置，其中 k 表示蛇的长度。蛇头位于 (x_1, y_1)，蛇尾位于 (x_k, y_k)，且相邻的身体部分位于共享一条边的格子中。

初始时，蛇的长度为 1，位于给定的格子 (r_s, c_s)。

有 4 种指令可以移动蛇：

- `U`：让蛇向上移动一步。蛇头将移动到 $(x_1 - 1, y_1)$。
- `D`：让蛇向下移动一步。蛇头将移动到 $(x_1 + 1, y_1)$。
- `L`：让蛇向左移动一步。蛇头将移动到 $(x_1, y_1 - 1)$。
- `R`：让蛇向右移动一步。蛇头将移动到 $(x_1, y_1 + 1)$。

当蛇头移动时，身体的每个部分也会相应移动。具体地，身体的第 i 部分 $(2 \le i \le k)$ 会移动到第 $(i-1)$ 部分在指令执行前所在的位置。同时，蛇尾原先占据的格子变为空格。

蛇不能移出网格。此外，蛇不能与自身碰撞——您必须保证在任何指令执行后，身体的任意两部分不会占据同一个格子。考虑如下边界情况：蛇头位于 (x_1, y_1)，蛇尾位于 (x_k, y_k)。如果蛇头接下来要移动到 (x_1', y_1')，则允许 $(x_1', y_1') = (x_k, y_k)$：我们可以想象一个现实场景中，蛇头移入该格子的同时蛇尾恰好移出。类似地，当 $k = 2$ 时，允许通过一条指令交换蛇头和蛇尾的位置。

有 $(nm - 1)$ 个苹果会依次出现。每次，一个苹果出现在某个当前未被蛇占据的格子 (r, c) 上。您需要输出一串指令，将蛇头移动到苹果所在的位置。蛇头必须恰好在最后一条指令执行后到达 (r, c)，并且在此之前不能进入 (r, c)。

对于您输出的每串指令，除最后一条外都是普通移动：蛇尾如上所述腾出其所在格子。最后一条指令是进食移动：当蛇头到达 (r, c) 时，蛇吃掉苹果。在这次移动中，蛇尾不会腾出其所在格子，因此蛇的长度增加 1。

您的任务是成功吃掉所有 $(nm - 1)$ 个苹果。

Input

有多组测试数据。第一行输入一个整数 T $(1 \le T \le 100)$，表示测试数据组数。对于每组测试数据：

第一行输入四个整数 n，m，r_s 和 c_s $(2 \le n, m \le 50,\ nm \le 100,\ 1 \le r_s \le n,\ 1 \le c_s \le m)$，表示网格的大小和蛇的初始位置。

Interaction Protocol

您需要依次吃掉所有 $(nm - 1)$ 个苹果。对于每个苹果：

- 首先读入一行两个整数 r 和 c $(1 \le r \le n,\ 1 \le c \le m)$，表示苹果的位置。保证苹果的位置当前未被蛇占据。

- 然后输出一个由字符 U、D、L、R 组成的字符串——将蛇头从当前位置移动到 (r, c) 的指令序列。为了增加难度，我们限制字符串的长度不超过 nm。

每输出一行后，不要忘了刷新输出缓冲区。可以证明答案总是存在的。

Example

standard input	standard output
2 2 3 1 2 2 1 1 3 2 1 1 1 1 2 2 2 1 1 2 2 1 1 1 2	 LD URR DLL U R RD LU R

Note

下图展示了第一组样例数据吃掉 3 个苹果后的情况。

